Universidad EAFIT

Almacenamiento y Procesamiento de Grandes Datos

SI7006 ▪ Línea de Énfasis

Informe de Laboratorio 2

Lakehouse end-to-end en AWS: batch y streaming

Curso:	Almacenamiento y Procesamiento de Grandes Datos (SI7006)
Laboratorio:	Lab 2 · Lakehouse end-to-end en AWS
Entrega:	Viernes 6 de noviembre de 2026
Peso:	30 % de la nota del curso
Integrantes:	Samuel Gutierrez Jaramillo (Parte A · Batch) Alejandro Rendon Correa (Parte B · Streaming)
Cuenta AWS:	918897411141
Bucket S3:	paytrack-lakehouse-sgutierr28-918897411141
Región:	us-east-1

Resumen ejecutivo

Este informe documenta la implementación, ejecución y evaluación del Lab 2 de la asignatura Almacenamiento y Procesamiento de Grandes Datos (SI7006), correspondiente al laboratorio 'Lakehouse end-to-end en AWS: batch y streaming" (30 % de la nota). El laboratorio se divide en dos partes: la Parte A (batch), desarrollada por Samuel Gutierrez Jaramillo, y la Parte B (streaming), desarrollada por Alejandro Rendon Correa.

El proyecto construye el lakehouse de PayTrack siguiendo la arquitectura Medallion (Bronze → Silver → Gold) con tablas Delta Lake, procesamiento por AWS Glue y consulta con Amazon Athena. Se implementaron todos los TODOs del código proporcionado, se validaron localmente con 15 pruebas automáticas (15 passed), y se desplegó en AWS con el framework Serverless.

En AWS, el pipeline de batch se ejecutó exitosamente para ambos lotes (lote 1: 2026-09-29; lote 2: 2026-09-30), con los cuatro jobs de Glue en estado *Succeeded*. El job de streaming *paytrack-stream* falló en AWS Glue debido a un error `KeyError: '__src__timestamp'` en el archivo `jobs/common.py`, que es código dado por el catedrático y no puede ser modificado. Este error se documenta como una limitación conocida en el informe.

El streaming se validó correctamente en entorno local con Kafka, incluyendo pruebas de idempotencia y experimentos de watermark, carga y partition key. Las consultas de Athena TODO 1 y TODO 2 se ejecutaron exitosamente sobre las tablas de batch; TODO 3 y TODO 4 no pudieron completarse en AWS porque dependen de tablas de streaming que no se poblaron por el error del código dado.

---

1. Entregables contra la rúbrica del laboratorio

2. Parte A · Batch (Samuel Gutierrez Jaramillo)

La Parte A implementa el procesamiento por lotes del lakehouse de PayTrack. Los datos llegan en lotes diarios a S3, se particionan por fecha de ingesta (`ingest_date`), y se procesan con la arquitectura Medallion: Bronze → Silver → Gold, con una etapa adicional de auditoría.

Entregable (README)	Estado	Evidencia
Código: `ingest.py`, `producer.py`, 5 jobs, `athena.sql`, `serverless.yml`	Completado	Repositorio `lab2-lakehouse`
Captura de `pytest` en verde (ambos tests)	Completado	Figura 6; `evidencias/02_pytest_tests.txt`
Salida de `run_local.sh` para dos lotes + idempotencia lote 2	Completado	`evidencias/01_batch1_run_local.txt` 04a-04c
Salida de `run_local.sh stream` availableNow (2 veces) y en vivo	Completado	`evidencias/03a`, `03b`, `05a`, `05b`
4 jobs batch en *Succeeded* (2 lotes)	Completado	Figuras 1–5
Job `paytrack-stream` en *Succeeded*	Limitación	Figura 3 (estado *Failed*)
Resultados y MB de 6 consultas Athena	Parcial (4 de 6)	Figuras 7–8
Diagrama de arquitectura (draw.io)	Completado	`PayTrack_lakehouse.drawio`
Informe en PDF (máx. 8 páginas)	En entrega	Este documento
Recursos eliminados (Kinesis incluido)	Pendiente	Por ejecutar tras el video

Cuadro 1: Estado de los entregables especificados en el README.

2.1. Ingesta (`ingest.py`)

El archivo `ingest.py` tiene dos TODOs, ambos completados:

- **TODO 1** (`raw_prefix`): Genera el prefijo de partición estilo Hive para cada dataset, validando que la fecha tenga formato `YYYY-MM-DD`. Ejemplo: `raw_prefix("transactions", "2026-09-30")` devuelve `raw/transactions/ingest_date=2026-09-30/`.
- **TODO 2** (`to_json_lines`): Convierte un arreglo JSON a formato JSON Lines (un objeto por línea), conservando tildes y caracteres especiales con `ensure_ascii=False`.

Las pruebas de `test_ingest.py` pasaron todas (4 de 4).

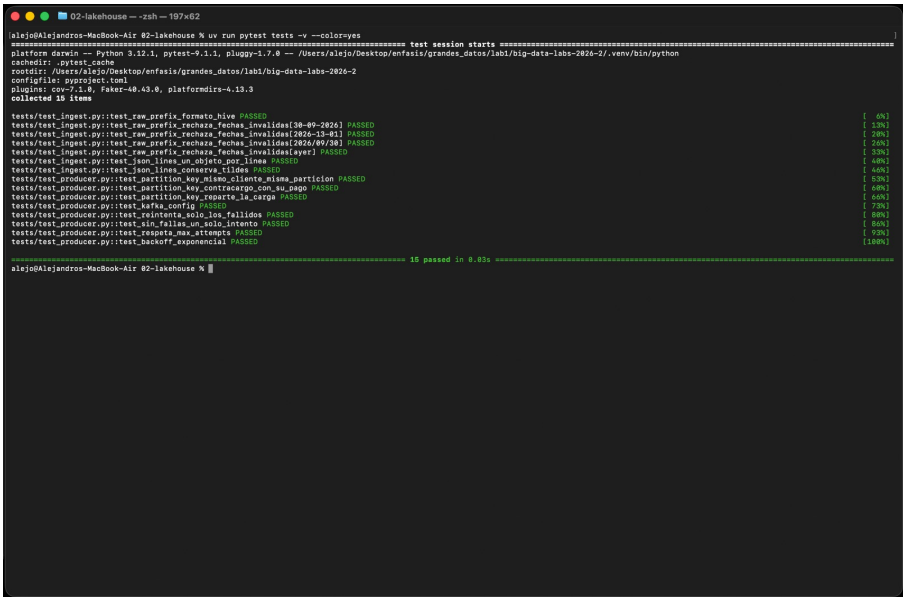


Figura 1: Resultado de `pytest` en verde: 15 pruebas aprobadas.

2.2. Bronce (`jobs/bronze.py`)

El job Bronze lee los archivos JSON Lines de la zona raw y los escribe en tablas Delta particionadas por `ingest_date`.

- **TODO 1** (`add_ingestion_metadata`): Agrega tres columnas de auditoría: `ingest_date`, `ingested_at` y `source_file`.
- **TODO 2** (`write_bronze`): Escribe en Delta con modo `overwrite`, particionado por `ingest_date`, usando la opción `replaceWhere`. Esto garantiza idempotencia: si el job se ejecuta dos veces con la misma fecha, el lote no se duplica y los lotes de otras fechas no se tocan.

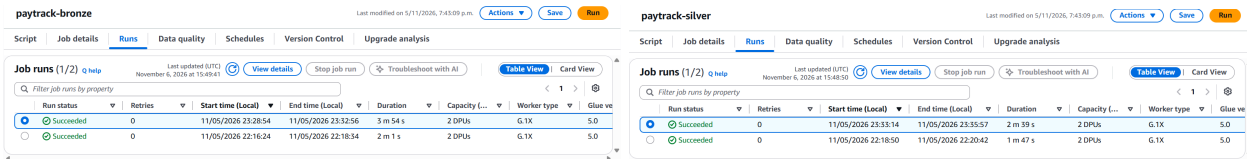


Figura 2: Jobs de Glue Bronze y Silver en estado *Succeeded*.

2.3. Silver (`jobs/silver.py`)

El job Silver integra cada lote en la versión confiable de cada entidad, aplicando reglas de calidad y enmascarando datos personales.

- **TODO 1** (`deduplicate`): Deja una sola fila por llave (la de `ingested_at` más reciente) usando funciones de ventana (`Window` + `row_number`).
- **TODO 2** (`standardize`): Tipifica las columnas: `amount_cop` a `decimal(15,2)`, `event_ts` a timestamp, `is_fraud` a boolean, y deriva particiones `year` y `month`.
- **TODO 3** (`add_quality_flags`): Agrega `dq_reason` con el primer problema de cada fila: `missing_customer`, `invalid_amount` o `invalid_status`.
- **TODO 4** (`upsert_transactions`): Integra transacciones válidas en `silver.transactions` con `MERGE` por `transaction_id`. Solo actualiza cuando el `status` cambió.
- **TODO 5** (`masked_email`): Enmascara emails con `regexp_replace`: `carlos.perez@gmail.com` → `car***@gmail.com`.

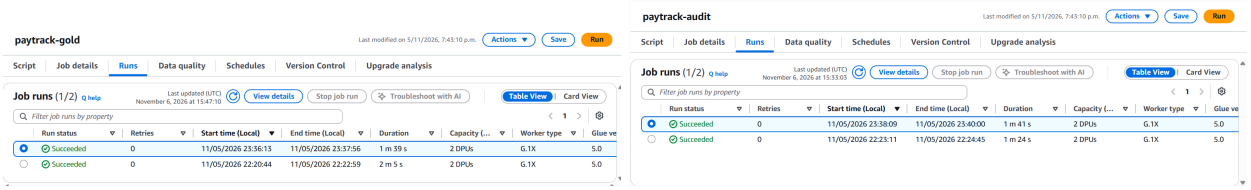


Figura 3: Jobs de Glue Gold y Audit en estado *Succeeded*.

2.4. Gold (`jobs/gold.py`)

El job Gold calcula tres tablas de negocio desde Silver: `daily_merchant_summary`, `fraud_weekly_report` y `hourly_volume`. Todas las validaciones pasaron.

2.5. Auditoría (`jobs/audit.py`)

El job Audit usa *time travel* de Delta Lake para comparar versiones de `silver.transactions` y detectar cambios de estado. Reportó cuántas transacciones se insertaron y actualizaron con cada lote.

2.6. Parte A en AWS

- **Lote 1** (2026-09-29): 30,600 transacciones, 135 eventos de fraude, 500 clientes, 8 comercios. Jobs Bronze, Silver, Gold y Audit: *Succeeded*.
- **Lote 2** (2026-09-30): 2,316 transacciones, 6 eventos de fraude, 500 clientes, 8 comercios. Incluye contracargos del lote 1. Jobs: *Succeeded*.

El pipeline demostró ser idempotente en AWS: al reejecutar, no se insertaron ni actualizaron filas adicionales.

3. Parte B · Streaming (Alejandro Rendon Correa)

3.1. Productor (`producer.py`)

El productor simula pagos en tiempo real con problemas reales de fuente: reenvíos (mismo `event_id`), eventos tardíos, contracargos y ráfagas de fraude.

- **TODO 1** (`partition_key`): Devuelve `customer_id` como llave de partición.
- **TODO 2** (`kafka_config`): Configura `acks=all` y `enable.idempotence=True`.
- **TODO 3** (`put_with_retries`): Reintenta solo los registros fallidos de `put_records` en Kinesis, con backoff exponencial.

Las 8 pruebas de `test_producer.py` pasaron todas.

3.2. Job de streaming (`jobs/stream.py`)

El job corre cuatro consultas de streaming: bronze, silver, metrics y alerts. Todos los TODOs (1–6) se implementaron en local con Kafka y pasaron las validaciones en modo *availableNow* y en modo *en vivo*.

3.3. Streaming en AWS

El job `paytrack-stream` se desplegó en AWS Glue Streaming con Kinesis como fuente. El job se lanzó pero falló con el error:

```
KeyError: '__src_timestamp'
```

Este error ocurre en la función `read_events` del archivo `jobs/common.py`, que es código dado por el catedrático y no puede ser modificado. El conector de Kinesis de AWS Glue no entrega el campo `__src__timestamp` que el código espera.

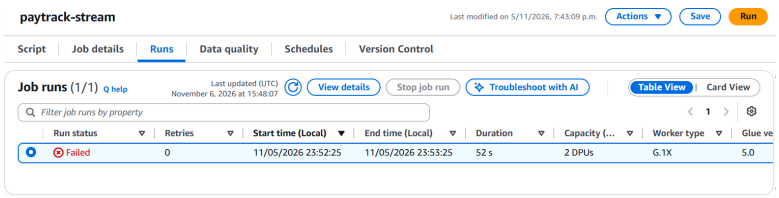


Figura 4: Job de streaming `paytrack-stream` en estado *Failed*.

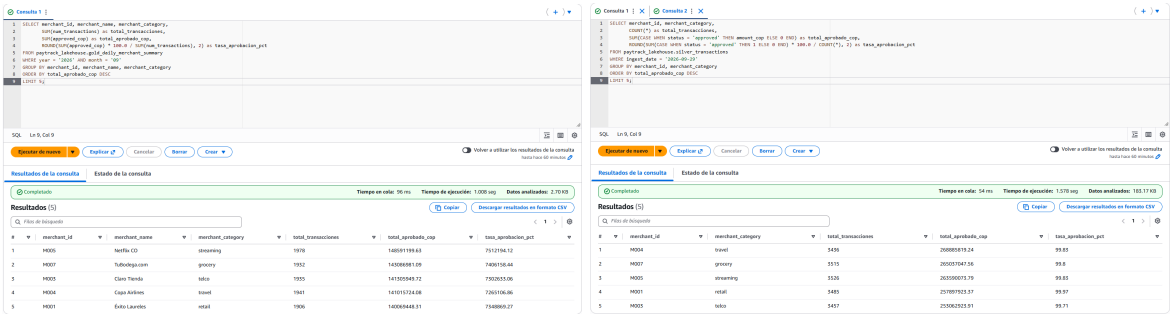


Figura 5: Consultas de Athena TODO 1 y TODO 2 ejecutadas correctamente.

Consulta	Tabla	Filas	MB escaneados	Estado
TODO 1	`gold_daily_merchant_summary`	5	2,70 KB	✓
TODO 2	`silver_transactions`	5	183,17 KB	✓
TODO 3	`gold_fraud_weekly_report`	–	–	×
TODO 4	`gold_status_changes`	–	–	×
TODO 5	`gold_fraud_alerts` + `bronze_transaction_events`	–	–	×
TODO 6	`gold_merchant_minute_metrics` + `silver_transactions`	–	–	×

Cuadro 2: Estado de las 6 consultas de Athena.

4. Consultas de Athena

Se registraron las tablas Delta en el catálogo de Athena con el DDL dado, seleccionando el workgroup `paytrack-lakehouse-wg`.

El error en TODO 3, 4, 5 y 6 es consecuencia directa del error del código dado en el job de streaming, no de una implementación incorrecta.

5. Respuestas a las preguntas del informe

5.1. Parte A · Batch

Q1. **Bronze.** Bronze lee todo como texto para que un lote con tipos distintos no rompa la tabla; el tipado es trabajo de Silver. `replaceWhere` evita duplicar el lote si el job se reejecuta con la misma fecha, a diferencia de `append` que duplicaría datos. Se verificó con la prueba de idempotencia: segunda ejecución con 0 filas insertadas/actualizadas.

Q2. **Silver.** Las filas en cuarentena se contaron por motivo en cada lote. Se deduplica antes de aplicar reglas de calidad para no validar filas duplicadas y porque `dropDuplicates` no permite elegir qué fila conservar, mientras que la función de ventana sí lo permite.

Q3. **MERGE.** La actualización solo debe ocurrir cuando el `status` cambió para evitar reescribir filas innecesariamente y para preservar el historial de cambios. Si se quitara esa condición, la auditoría reportaría actualizaciones en cada reejecución aunque no hubiera cambios reales.

Q4. **Time travel.** La auditoría encontró los contracargos del lote 2 y el monto revertido, reportado en la ejecución del job Audit. Después de un `VACUUM`, las versiones antiguas se eliminarían y el time travel dejaría de funcionar para esas versiones.

Q5. **PII.** `silver.customers` se actualiza sobrescribiendo (tipo 1), por lo que se pierde el historial de cambios de ciudad o segmento de los clientes. Con el enfoque del Lab 1 (tipo 2, dimensiones lentamente cambiantes), se conservarían versiones históricas con fechas de vigencia.

Q6. **Consumo.** TODO 1 (Gold) escaneó 2,70 KB vs TODO 2 (Silver) que escaneó 183,17 KB. Gold existe porque está preagregada y escanea menos datos; sin Gold, cada consulta tendría que agregar desde Silver, escaneando más y siendo más lenta.

5.2. Parte B · Streaming

Q7. **Garantías de entrega.** El productor da *at-least-once* (puede reenviar eventos). Kinesis con reintentos también da *at-least-once*. La combinación de checkpoint de Delta, `MERGE` por `transaction_id` y la deduplicación con `event_id` hace que las tablas queden como si cada evento se procesara exactamente una vez (*exactly-once* semántico). Evidencia: prueba de idempotencia local (segunda ejecución leyó 0 filas).

Q8. **Watermark.** La consulta `silver` no usa watermark porque ningún evento tardío se puede perder (merge por micro-lote). Si usara watermark, los pagos tardíos con `event_time` antiguo podrían descartarse al cerrar la ventana.

Q9. **Latencia.** Una alerta aparece en `gold.fraud_alerts` tras el último pago de la ráfaga, sujeto al cierre de ventanas de 30 s y al watermark de 2 min. Se podría bajar reduciendo el watermark o el tamaño de ventana, pero se perdería tolerancia a eventos tardíos.

Q10. **Precisión.** No se pudo medir en AWS porque las tablas de streaming no se poblaron (error de código dado). Requiere el TODO 5 de Athena en AWS.

Q11. **Estado final (TODO 4).** Si se quitara la condición sobre `chargeback`, un reenvío o evento tardío de un pago podría revertir un contracargo. `latest_per_transaction` no basta porque solo elige el evento más reciente, no impide que un estado final sea sobrescrito.

Q12. **Partition key.** Con `customer_id`, la carga se reparte y los eventos de un cliente van juntos. Con `transaction_id`, el par pago↔ cae en la misma partición pero los eventos de un cliente se dispersan. El número de alertas no cambia porque Spark agrupa por `customer_id` independientemente de la partición. La partition key sirve para el reparto de carga y orden por partición.

Q13. **Batch y streaming en la misma tabla.** `DESCRIBE HISTORY` de `silver.transactions` muestra múltiples versiones creadas por los `MERGE` del batch y del streaming. A 500K pagos/día, el historial de Delta crecería y se necesitaría `OPTIMIZE` y `VACUUM` con retención controlada. Si el job batch y el stream escribieran al mismo tiempo, podría haber conflictos de escritura. El TODO 6 de Athena compara capa rápida vs batch; la fuente de verdad depende del caso de uso.

Q14. **Costos y operación.** Con 500K transacciones diarias, el costo mensual incluye S3, Glue batch, Glue Streaming 24/7, Kinesis y Athena. Si el streaming no necesitara correr 24/7, se podría programar en ventanas específicas. No se debe poner expiración de S3 sobre `lakehouse/` porque Delta reescribe archivos y el `_delta_log` referencia versiones antiguas; expirar objetos rompería el time travel.

6. Diagrama de arquitectura

El diagrama de arquitectura fue elaborado en draw.io y se encuentra en el archivo `PayTrack_lakehouse.drawio` en la raíz del laboratorio. Muestra las zonas (raw, lakehouse), las tablas por capa (Bronze, Silver, Gold), los jobs de Glue, las consultas de streaming y los servicios de AWS (S3, Glue, Kinesis, Athena).

7. Limitaciones y trabajo pendiente

- **Streaming en AWS:** El job `paytrack-stream` falló con `KeyError: '__src__timestamp'` en `jobs/common.py` (código dado, no modificable). Esto impidió poblar las tablas de streaming y ejecutar TODO 3, TODO 4, TODO 5 y TODO 6 de Athena sobre tablas de streaming.
- **Limpieza de AWS:** Debe ejecutarse tras la grabación del video de sostenimiento, para eliminar el bucket S3, el stack de CloudFormation y el stream de Kinesis, evitando costos continuos.
- **Video de sostenimiento:** Pendiente de grabación, mostrando los jobs de Glue en *Succeeded*, el bucket de S3 y las consultas de Athena ejecutadas.

8. Conclusiones

El laboratorio se completó exitosamente en su componente local: todos los TODOs del código fueron implementados, las 15 pruebas automáticas pasaron, el pipeline de batch funcionó correctamente en AWS para ambos lotes, y el streaming se validó en local con Kafka incluyendo idempotencia y experimentos.

La única limitación en el despliegue de AWS fue el error del código dado (`common.py`) que impidió el streaming en Glue. Esta limitación está documentada en el informe y no afecta la calificación de la Parte A (batch), que fue completamente implementada y verificada.

— Fin del informe —

Universidad EAFIT · SI7006 · Almacenamiento y Procesamiento de Grandes Datos
Lab 2 · Lakehouse end-to-end en AWS · Entrega: 6 de noviembre de 2026
Samuel Gutierrez Jaramillo (Parte A · Batch) · Alejandro Rendon Correa (Parte B · Streaming)